# Three Positions, Not Four: Composing CKS Substrates with RAG Indexes, Fine-Tuned Models, Vector Databases, and External Knowledge Stores

**Author:** Wenxin Li (Independent Researcher)
**ORCID:** 0009-0004-8065-3235
**Date:** 30 April 2026


---

## Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, how CKS substrates compose with adjacent AI components — RAG retrieval indexes, fine-tuned LLMs, vector databases, and external structured knowledge stores — without violating the architectural commitments the source paper defends.

## Abstract

Real CKS deployments rarely sit alone. A regulated coordination workflow with a CKS substrate at its center is typically embedded in a larger AI system that includes one or more retrieval indexes over source documents, fine-tuned models specialized to a domain, vector databases, and external structured stores that predate or postdate the substrate. The source paper distinguishes CKS from each of these on architectural grounds — RAG at §1.1 and §2.3, parametric memory and fine-tuning at §6.2, external structured memory at §6.2 and §8.2 — but does not separately formalize how the distinct objects coexist when a deployment uses both. This note formalizes the answer. The source paper supports exactly three positions an adjacent AI component can occupy relative to a CKS substrate — *as input to a cell* (Pattern A), *as a derived view of substrate content* (Pattern B), or *as a separate concern* (Pattern C) — and three corresponding anti-patterns into which a deployment drifts when no position is named explicitly. A hybrid deployment is CKS-coherent if and only if every adjacent AI component sits in one of the three positions. The note states each position's requirements, identifies the anti-patterns each preempts, and supplies an operational test for whether a hybrid deployment retains the source paper's commitments.

## 1. Why hybrid composition needs to be addressed as its own framework

The source paper distinguishes CKS from RAG (§1.1, §2.3), from parametric memory and fine-tuning (§6.2), and from external structured memory used as an LLM's primary substrate (§6.2, §8.2). A companion note in this series formalizes those three architectural distinctions as commitments about what CKS *is*. The present note answers the practical question those distinctions raise and that the source paper does not separately address: how do CKS substrates *coexist* with these objects when a real deployment uses both?

The question is not optional. Isolated CKS substrates with no adjacent AI components are the exception in real systems; hybrid deployments are the common case. A regulated coordination workflow may sit beside a RAG system retrieving from a corpus, a fine-tuned model specialized to a domain vocabulary, and one or more vector databases the organization already operates. The architecture must hold across that composition or its commitments hold only in degenerate configurations, which is not what the source paper claims. The source paper supports specific composition patterns implicitly across §2.3, §3.1, §4.1, and §4.2; this note formalizes those patterns and identifies their boundaries.

## 2. The three composition patterns

A given deployment may use any combination of three composition patterns. The architectural commitment is that each adjacent AI component is positioned in one of these patterns, not in a fourth pattern that would break CKS commitments.

**Pattern A — Adjacent component as input to a CKS cell.** An adjacent AI component (a RAG index over source documents, a fine-tuned LLM specialized to a domain, a vector database supporting semantic search, an external knowledge store) is consulted by a CKS cell during the cell's execution. The cell reads from the substrate as its source of truth, may also query the adjacent component for additional context, and writes its outputs back to the substrate under its orchestration rule. The adjacent component is part of the cell's reasoning environment; the substrate remains authoritative.

**Pattern B — Adjacent component as derived view of substrate content.** The substrate's content is indexed, embedded, or summarized into an adjacent component to support specific queries — a vector index for semantic search over substrate content, a derived knowledge graph, a search-optimized projection. The adjacent component is read as a derived view; it is not authoritative, and any conflict with the substrate is resolved in the substrate's favor.

**Pattern C — Adjacent component as separate concern.** The adjacent component handles a distinct concern that does not affect coordination state — for example, a RAG retrieval system that answers reference-document queries entirely outside the coordination work, or a fine-tuned LLM used for tasks the substrate does not represent. The two systems coexist without architectural coupling.

The patterns are positions in the architecture, not deployment recipes. A single deployment may use all three at once: a RAG index over reference documents may be consulted by some cells under Pattern A while answering ad-hoc reference queries that are out of coordination scope under Pattern C; a vector index may simultaneously supply semantic search over substrate content as a derived view under Pattern B. What the architecture requires is that for each adjacent component, the position it occupies is identifiable and the requirements of that position are met.

## 3. What Pattern A requires of the substrate–component boundary

For a CKS cell to use an adjacent component as input without violating the substrate-as-source-of-truth commitment (§3.1, §4.1) or AI-as-substrate-mediator (§4.2), three properties must hold.

*Provenance crosses the component boundary cleanly.* The cell's writes back to the substrate must be attributable to the cell, the orchestration rule that authorized the write, and (where applicable) the adjacent components consulted. Path retraceability — the source paper's commitment that the substrate carries the trace of how decisions came to be (§3.1, §5) — does not stop at the cell boundary; it extends through

whatever the cell consulted. A write produced with input from a RAG index must record that consultation as substrate provenance; otherwise the retraceable path breaks at the moment the cell's reasoning crossed an opaque boundary.

*The substrate is authoritative on coordination questions.* The cell's reasoning may draw on the adjacent component for additional context, source-document excerpts, or background information; it cannot defer to the adjacent component on questions the substrate is the source of truth for. If the cell reads from a RAG index and from the substrate and the two disagree on a coordination question — what was decided, by whom, under what authority, with what rationale — the substrate wins by definition.

*The cell's outputs remain orchestration-rule-governed.* The adjacent component may inform the cell's reasoning, but the cell's writes are still subject to the rule that authorizes them. An adjacent component does not provide additional write authorization, does not bypass the orchestration rule, and does not extend the cell's authority beyond what the rule defines. The pattern preserves AI-as-substrate-mediator so long as the cell continues to read from and write to the substrate under its rule; the adjacent component is part of the cell's reasoning environment, not a substitute for the substrate.

## 4. What Pattern B requires of the derived-view boundary

For an adjacent component to function as a derived view without violating the substrate-as-source-of-truth commitment, three properties must hold.

*The view is regeneratable from substrate state.* The component's content must be derivable from the substrate at any time. If substrate content changes, the derived view is updated; if the derived view drifts, the substrate is the reference and the view is rebuilt. A derived view that has accumulated content the substrate cannot regenerate has stopped being a derived view and become an independent store the deployment now has to govern as substrate, which is not the position the architecture supports.

*The view is non-authoritative on coordination questions.* Reads from the derived view that affect coordination decisions must be reconciled against the substrate before being treated as authoritative. The derived view is for performance or convenience — semantic search, summarization, or faster lookup along access paths the substrate's native shape does not optimize for. Correctness on coordination questions comes from the substrate, not from the projection.

*Writes do not happen to the derived view.* When a cell writes coordination state, it writes to the substrate, and the derived view is updated downstream. A pattern in which cells write to the derived view and the substrate is reconstructed from the view inverts the source-of-truth commitment, and the architecture does not permit that inversion. If the deployment finds itself writing to the derived view because the substrate's interface is inconvenient, the substrate's interface needs work; the source of truth does not migrate. The pattern preserves substrate-as-source-of-truth so long as the derived view is treated as derived; treating it as authoritative shifts the source of truth out of the substrate.

## 5. What Pattern C requires of the separate-concern boundary

For an adjacent component to coexist as a separate concern, three properties must hold.

*The component does not hold coordination state.* It may handle non-coordination work — document retrieval, semantic search over reference material, domain-specialized inference, language tasks the

substrate does not represent — but the categories of state the substrate is the source of truth for must not migrate to the component. A reference-document retrieval system that answers “find me passages about topic X” without recording what was decided about topic X is Pattern C; the same system used to answer “what did we decide about topic X” has crossed into coordination scope and must be evaluated under Pattern A.

*The component does not exercise governance authority.* If the component is involved in coordination decisions, its involvement must be modeled as Pattern A, not as autonomous decision-making. A fine-tuned model used to draft text the substrate then carries is acting as input to a cell; a fine-tuned model writing decisions into substrate fields under no orchestration rule is exercising governance authority the architecture does not grant.

*The boundary between the component's domain and the substrate's domain is inspectable.* A human exercising the inspect right (§3.1, §3.3) must be able to determine which decisions the substrate carries and which the adjacent component handles, without ambiguity. If the boundary is not inspectable, the adjacent component is in practice outside human governance, which violates the source paper's first commitment regardless of whether the component happens to behave well. The pattern preserves the architectural commitments by limiting the adjacent component's scope to non-coordination concerns; coordination work that crosses into the adjacent component's scope must be modeled under Pattern A.

## 6. Anti-patterns that break CKS commitments

Three patterns look like hybrid composition but violate the architecture. They are named here because deployments drift into them when no composition position is identified explicitly, and a deployment that has not named the patterns has no frame for recognizing the drift.

*Adjacent component as substitute for substrate.* The deployment treats a RAG index, a vector database, or an external store as if it were the source of truth for coordination questions, gradually replacing or competing with the substrate. The drift typically begins as a Pattern B derived view that becomes faster or more convenient than the substrate it derives from, then becomes the surface participants reach for first, then becomes the place new content is written. By the time the substrate is being reconstructed from the “derived” view, the source of truth has migrated. The anti-pattern violates substrate-as-source-of-truth and produces, at the system level, the failure mode §6.2 names as context rot: substrate content participants thought authoritative has been compressed, summarized, or re-embedded out of fidelity with what the substrate carries.

*Adjacent component as ungoverned writer.* The deployment allows an adjacent component — typically an autonomous agent backed by a fine-tuned model — to write coordination state outside cell mediation and orchestration-rule authorization. The component's writes appear in the substrate without orchestration-rule trace, without provenance for the consultation that produced them, and without the authority constraints a cell would have applied. This violates AI-as-substrate-mediator (§4.2) and human-governed (§3.1, §3.3). The architectural test is simple: every substrate write must be traceable to a cell and its rule; writes that are not have come from outside the architecture.

*Hidden bidirectional coupling.* The substrate and an adjacent component update each other through automated processes that no human-authored orchestration rule governs. A typical instance: a vector index that reindexes substrate content automatically while also injecting embeddings back into substrate fields, with neither direction passing through a cell or a rule. The substrate–cell boundary §2.1 defends is silently

bypassed, and path retraceability breaks because the trace that would have recorded “this write happened because rule R authorized cell C” no longer exists. The fix is to bring each direction of update under an orchestration rule, which converts the coupling into Pattern A (cell consults the index, writes substrate under rule) and Pattern B (substrate change triggers index rebuild as a derived view).

The three anti-patterns share a common failure mode: an adjacent component has acquired a position the architecture does not name, and the commitments degrade where the position is unnamed. Naming the three legitimate positions is what makes the drift visible at the moment a practical decision is made.

## 7. How the patterns interact with adjacent CKS commitments

Each composition pattern preserves the source paper's commitments by maintaining specific properties.

*Substrate-as-source-of-truth* is maintained by Pattern A (the adjacent component is consulted but not authoritative), by Pattern B (the derived view is non-authoritative and regeneratable), and by Pattern C (the adjacent component handles non-coordination concerns by definition).

*AI-as-substrate-mediator* is maintained by all three patterns because adjacent components do not gain substrate-write authority. They inform reasoning (Pattern A), supply derived views (Pattern B), or handle separate concerns (Pattern C); none of these positions extends governance authority to the component.

*Path retraceability* is maintained as long as cell writes carry provenance for adjacent-component consultations. Pattern A makes the requirement explicit; Pattern B does not produce writes; Pattern C does not affect coordination state.

*Tool-agnosticism* is maintained because the substrate's host environment continues to satisfy the three minimal requirements (§7.1) regardless of what adjacent components are present. Adjacent components do not become required for substrate operation, and a deployment with no adjacent components remains a valid CKS deployment.

*Linear-cost scaling* (§6.1, §6.2) is maintained as long as adjacent components do not introduce size-proportional overhead the substrate inherits. A derived view whose update cost grows superlinearly with substrate size would push the system out of linear cost; this is a deployment concern hybrid compositions must manage rather than an architectural relaxation the patterns grant.

## 8. Operational test

A hybrid deployment instantiates the CKS architecture if and only if all of the following are true at all times during the deployment's existence:

1. Each adjacent AI component is positioned as Pattern A, Pattern B, or Pattern C; no component occupies a fourth position.
2. Coordination state lives in the substrate; adjacent components do not hold authoritative coordination state.
3. All writes to the substrate are mediated by cells under orchestration rules; adjacent components do not write coordination state directly.
4. The substrate's host environment continues to satisfy the three minimal requirements (§7.1) regardless of what adjacent components are present.

5. The provenance recorded for substrate writes includes the adjacent components consulted, where applicable, so the retraceable path crosses the component boundary cleanly.

A deployment that fails any of (1)–(5) may be a useful system, and may even be governed in some other sense, but is not CKS-coherent in the sense the source paper defends.

## 9. Why naming these patterns matters

Hybrid deployments are the common case in real AI systems, and implementations that do not name the composition patterns explicitly tend to drift into the anti-patterns. The reason is mechanical: the architectural commitments are not visible at the moment a practical decision is made. A team facing the question “should we let the agent write to the substrate directly to skip the cell?” needs a frame that surfaces the commitment the shortcut would violate. “Pattern A says the agent's input goes through the cell; the cell's rule authorizes the write” is that frame, and the proposed shortcut is identifiable as a Pattern A bypass. Without the frame, the same decision reads as a sensible engineering simplification and is taken without recognizing what it costs.

Naming the three positions also bounds the deployment's design space honestly. A team may discover that a desired composition fits none of the three positions cleanly, in which case the architecture is informing them that the desired composition is one the source paper does not support — not that the source paper has a gap for them to fill in by analogy. The architecture's silence on a fourth position is not an invitation to invent one; it is a constraint, and respecting the constraint is what makes hybrid composition defensible without introducing new commitments beyond the source paper.

---

## Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

## How to cite this note

Li, W. (2026). *Three Positions, Not Four: Composing CKS Substrates with RAG Indexes, Fine-Tuned Models, Vector Databases, and External Knowledge Stores.* 30 April 2026. ORCID: 0009-0004-8065-3235.